

UNIVERSITY OF LATVIA
FACULTY OF EXACT SCIENCES AND TECHNOLOGIES
COMPUTER SCIENCE DEPARTMENT

GOOGLE MAPS REVIEW SENTIMENT CLASSIFICATION
“LANGUAGE TECHNOLOGY BASICS” COURSE PROJECT

Authors:

Aleksandra Dmitruka, ad22069

Markuss Birznieks, mb20075

RIGA, 2025

Contents

1.	Introduction	3
1.1.	Project goal.....	3
1.2.	Connection to language technologies.....	3
1.3.	Existing solutions to given task	4
1.4.	Possible usage of our results.....	4
2.	Our approach	5
2.1.	Models and training used.....	5
2.2.	Our dataset, splitting and labelling methods, analysis	6
2.2.1.	Sampler approach	7
2.2.2.	Full dataset approach.....	8
2.2.3.	25/5/5 split approach.....	10
2.2.4.	3-classes approach	11
2.3.	Metrics and optimization solutions used	13
2.3.1.	Metrics for model evaluation.....	13
2.3.2.	Text processing.....	14
2.3.3.	Training and dataset flags	14
2.3.4.	Tokenization optimization.....	15
2.3.5.	Parallel training using local machines and Kaggle.....	15
3.	Results	16
3.1.	Sampler approach	16
3.2.	Full dataset approach.....	20
3.3.	25/5/5 split.....	24
3.4.	3-class approach.....	28
3.5.	Extra testing	31
4.	Conclusions	32

1. INTRODUCTION

1.1. Project goal

Our goal is to evaluate the usefulness of additional training of a BERT model for sentiment analysis of Google Maps reviews, in the context of comparing three types of models:

- The original BERT model, fine-tuned for review rating prediction (from 1 to 5)
- The additionally trained model, based on the original one
- The DistilBERT model, used as lighter analog of pre-trained original BERT
- A simple self-designed model (LSTM), which provides the same classification functionality as the original model

As an addition, our goal is also to compare different dataset splitting and labeling methods as a way to deal with noisy user-generated data

1.2. Connection to language technologies

Text classification gets on to being the basis of this work, being one of the most important fields of language technology. Our project is related to language technology development in the following ways:

- **Textual data processing:** we utilize essential NLP steps such as text analysis, filtering, tokenization, and vectorization. These preprocessing stages are needed for converting raw text into formats suitable for machine learning models and are widely used across various language technology tasks.
- **Evaluation metrics and testing protocols:** Choosing appropriate metrics such as F1 score, off-by-one accuracy and maintaining consistent evaluation procedures are critical for comparing different models. This ensures reliable results of performance, which is a central concern in language technology research
- **Model comparison and optimization:** By comparing advanced models like BERT with simpler architectures such as LSTM and lighter versions of the same model like DistilBERT, the project explores different approaches to sentiment classification. This contributes to understanding how best to enhance language technologies for specific tasks like review rating prediction.
- **Dealing with imbalanced and noisy textual data:** user-generated reviews can be highly imbalanced by mood, with varying lengths and levels of detail. Language technologies must be robust enough to handle such challenges, and our model comparison addresses this practical aspect.

1.3. Existing solutions to given task

When looking at global solutions for sentiment analysis, several approaches stand out:

- Ready-made sentiment analysis models like bert, DistilBERT, and roberta, along with their fine-tuned versions. For example, DistilBERT-base-uncased-finetuned-sst-2-english and twitter-roberta-base-sentiment are widely used due to their strong performance on various text sentiment classification tasks.
- Services and APIs for text analysis, such as google natural language api, aws comprehend, and microsoft azure text analytics, offer accessible tools that simplify integration of sentiment analysis into applications without needing to train models from scratch.
- Numerous kaggle projects provide valuable datasets and benchmarks for review sentiment analysis, including amazon reviews, steam reviews, and google maps reviews, which help researchers and developers test and improve their models.
- Academic research papers explore additional training techniques and their general effectiveness for tasks like google maps review sentiment analysis, offering insights into methods that can boost model accuracy and robustness; this includes various [student projects](#) and [in-depth studies](#) on transfer learning and domain adaptation.

1.4. Possible usage of our results

The outcomes of our project, which evaluates the impact of additional training on BERT and other models, have practical implications in various real-world scenarios:

- **Improved customer feedback analysis for businesses:** By providing more accurate sentiment predictions on user reviews, businesses can better understand customer satisfaction and identify specific areas for improvement. This enables targeted service enhancements, reputation management, and personalized marketing strategies.
- **Enhanced review filtering and content moderation:** Platforms hosting user-generated reviews can implement these models to automatically detect low-quality or misleading feedback, improving the overall reliability and usefulness of reviews for future users.
- **Efficient resource allocation with lighter models:** The inclusion of the DistilBERT model as a lighter alternative offers opportunities for deploying sentiment analysis in environments with limited computational resources, such as mobile devices or real-time applications, without significant loss of accuracy.
- **Robust handling of noisy data:** By exploring different dataset splitting and labelling strategies, the project contributes methods to better manage the inherent noise and inconsistencies typical in user-generated content

2. OUR APPROACH

2.1. Models and training used

In this project, we used three types of models to explore different approaches to sentiment analysis of Google Maps reviews:

- **bert-base-multilingual-uncased-sentiment (fine-tuned by freezing all layers except the classification layer):** This model was selected because it is a robust, pre-trained transformer capable of understanding 1–5-star review classification of reviews and achieving 90% of off-by-one accuracy, which is important for diverse review datasets. By unfreezing the classification layer during fine-tuning, we aimed to preserve the general sentiment understanding encoded in the pre-trained layers while adapting the model efficiently to our specific task. This approach reduces the risk of overfitting and high recourse using, allowing for effective learning with limited task-specific data.
- **DistilBERT-base-uncased (trained from scratch):** DistilBERT is a smaller, distilled version of BERT that retains much of BERT’s language understanding capabilities while being more computationally efficient. We trained this model purely on our dataset to evaluate whether a lighter model could achieve comparable performance to full-scale finetuned BERT, making it a good candidate for real-world applications where computational resources or latency constraints are critical.
- **custom LSTM model (trained from scratch):** The Long Short-Term Memory (LSTM) network represents a classical recurrent neural network architecture widely used for sequential data processing. We included this custom model as a baseline to compare against transformer-based approaches. Despite lacking the contextual richness of transformers, LSTMs are still effective in capturing temporal dependencies in text and are computationally simpler and faster, providing look into the trade-offs between model complexity and performance. In our implementation, the model begins by converting input token indices into dense embeddings, allowing it to learn semantic representations of words. These embeddings are then passed through one or more LSTM layers, which maintain hidden and cell states across time steps to model the sequential structure of language. Finally, the output is passed through a fully connected linear layer to produce class scores for each token or sequence, enabling classification based on the learned temporal patterns.

```

class LSTM(nn.Module):
    def __init__(self, num_emb, output_size, num_layers=1, hidden_size=128):
        """
        Initializes an LSTM-based sequence model.

        Args:
            num_emb (int): Size of the vocabulary (number of unique tokens).
            output_size (int): Number of output classes (for classification).
            num_layers (int): Number of LSTM layers.
            hidden_size (int): Size of the hidden state vectors.
        """
        super(LSTM, self).__init__()

        # Embedding layer to convert token indices to dense vectors
        self.embedding = nn.Embedding(num_emb, hidden_size, padding_idx=0)

        # LSTM layer(s) to process the sequence of embeddings
        self.lstm = nn.LSTM(
            input_size=hidden_size,
            hidden_size=hidden_size,
            num_layers=num_layers,
            batch_first=True,
            dropout=0.5 # Applies dropout to outputs of each LSTM layer (except last)
        )

        # Fully connected layer to project the final hidden state to output size
        self.fc_out = nn.Linear(hidden_size, output_size)

    def forward(self, input_seq, hidden_in, mem_in):
        """
        Forward pass of the LSTM model.

        Args:
            input_seq (torch.Tensor): Input sequence of token indices (batch_size, seq_len).
            hidden_in (torch.Tensor): Initial hidden state(s) for the LSTM (num_layers, batch_size, hidden_size).
            mem_in (torch.Tensor): Initial cell state(s) for the LSTM (num_layers, batch_size, hidden_size).

        Returns:
            output_logits (torch.Tensor): Class scores for each token in the sequence (batch_size, seq_len, output_size).
            hidden_out (torch.Tensor): Final hidden state from the LSTM.
            mem_out (torch.Tensor): Final cell state from the LSTM.
        """
        # Convert input indices to embeddings
        input_embs = self.embedding(input_seq)

        # Pass embeddings through the LSTM
        output, (hidden_out, mem_out) = self.lstm(input_embs, (hidden_in, mem_in))

        # Project LSTM outputs to class logits
        return self.fc_out(output), hidden_out, mem_out

```

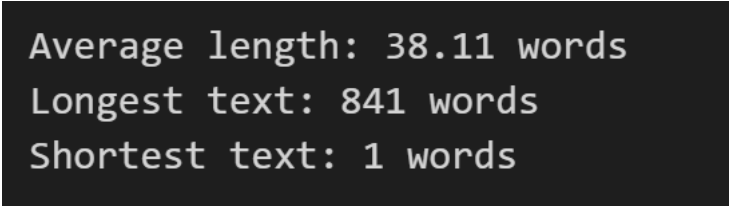
2.1.1. picture. Custom LSTM structure

2.2. Our dataset, splitting and labelling methods, analysis

Our dataset was collected manually using google-maps-review scrapper tool and a custom excel document with google maps links to different popular public places such as restaurants, sights, hotels. Four distinct strategies were implemented to evaluate model robustness and to explore different generalization patterns within the data. Tree of them - sampler, full dataset and 25/5/5 split - were used for 5-star classification with bert, DistilBERT and lstm while 3-class approach provided an insight on the effectiveness of mapping 5-star labels to positive/negative/neutral classes with DistilBERT and lstm only. In the beginning the general dataset analysis was performed to understand the label distribution and data length:

Label	Count	Percentage
1	21395	15.81%
2	7186	5.31%
3	11268	8.33%
4	21622	15.98%
5	73812	54.56%
Total	135283	100.00%

2.2.1. picture. Label distribution in collected dataset



```
Average length: 38.11 words
Longest text: 841 words
Shortest text: 1 words
```

2.2.2. picture. Analysis of text length

This general analysis allowed us to conclude that we will indeed be dealing with imbalanced set and that we won't be able to fit all the text to our models and will need to truncate it. That's why we decided to stay on 128 token truncations to keep our solution robustness and try on different dataset splitting methods which can help our models learn faster and/or more effective, since we both were interested in speed and high precision and recall.

To follow the best principles of NLP and machine learning we always separated out data in 3 parts - train, validation and test. The splitting was ensured to be constant with fixed seed.

2.2.1. Sampler approach

In the sampler approach, a balanced sampling strategy was used to fight class imbalance in full dataset and ensure a fair distribution of samples across training phase. Specifically, a customized sampler was developed to draw an equal number of samples from each class during training. This helped in reducing bias toward majority classes and provided more stability during the model training process.

The use of WeightedRandomSampler allowed assigning weights inversely proportional to class frequencies, ensuring under-represented classes received adequate representation during learning. This approach is particularly helpful in scenarios with naturally skewed data distributions, where certain classes may dominate due to higher frequency.

```
def createSampler(labels, num_samples=25000, replacement=True):
    """
    Creates a WeightedRandomSampler to address class imbalance during training.

    Args:
        labels (List[int] or torch.Tensor): List of class labels for the dataset.
        num_samples (int): Number of samples to draw per epoch.
        replacement (bool): Whether sampling is done with replacement.

    Returns:
        WeightedRandomSampler: Sampler that can be passed to a DataLoader.
    """

    # Count number of samples per class
    class_sample_count = pd.Series(labels).value_counts().sort_index().values

    # Compute class weights: inverse of class frequency
    # This gives higher weight to underrepresented classes
    weights = 1. / torch.tensor(class_sample_count, dtype=torch.float)

    # Assign weight to each sample based on its class
    sample_weights = weights[labels]

    # Create sampler that draws samples with probability proportional to their weight
    sampler = WeightedRandomSampler(
        weights=sample_weights,
        num_samples=num_samples, # Total number of samples to draw per epoch
        replacement=replacement # If True, allows repeated sampling of the same item
    )

    return sampler
```

2.2.1.1. picture. Sampler code with comments

2.2.2. Full dataset approach

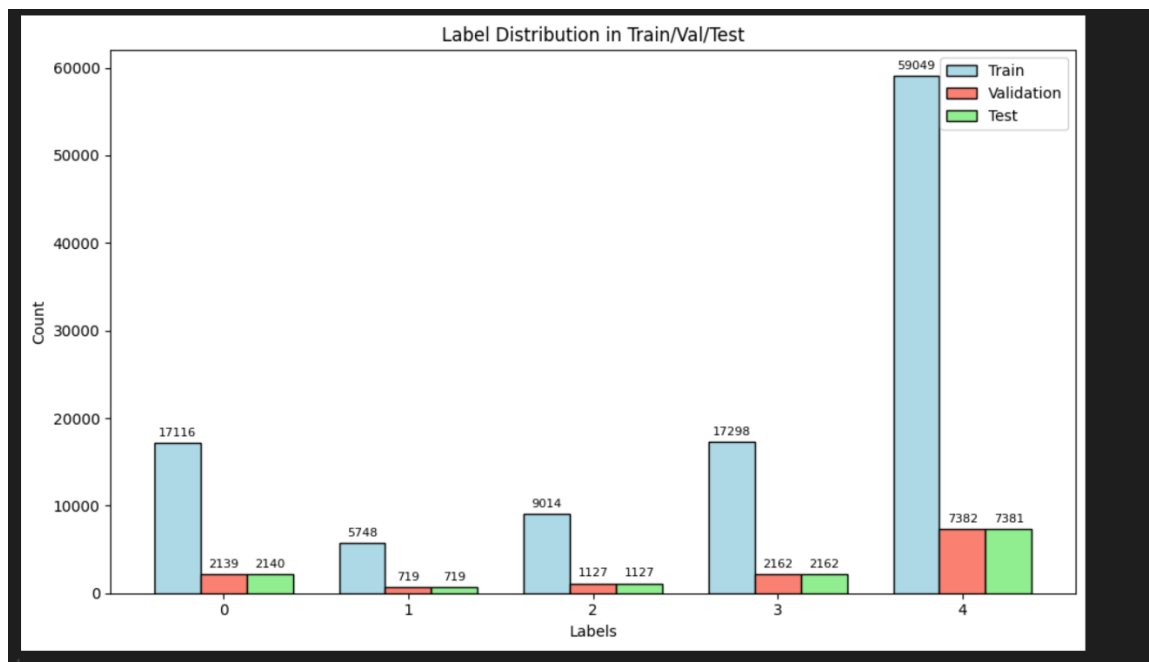
In the full dataset approach, the model was trained using the entirety of the available data without any artificial balancing or stratified sampling. The objective was to evaluate the model's behavior under a realistic distribution scenario, where some classes may naturally be underrepresented. It also was supposed to capture more features by analyzing all at once by our theory.

In the full dataset the distribution among train/validation and test remained the same as for sampler, e.g.:

```
Train size: 108225 (80.0%)
Validation size: 13529 (10.0%)
Test size: 13529 (10.0%)
```

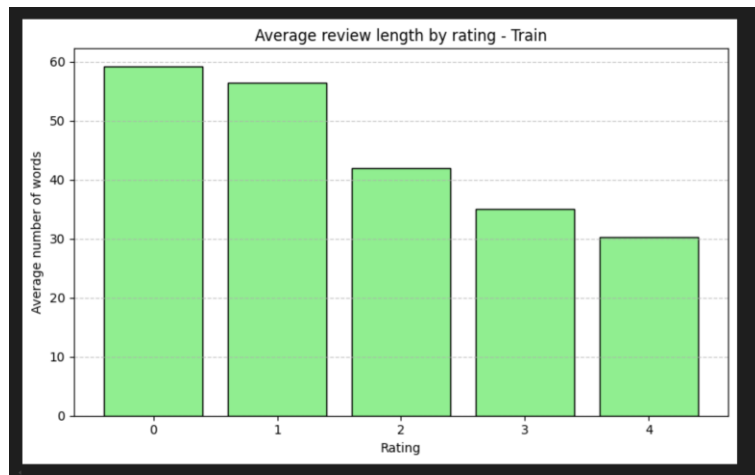
2.2.2.1. picture. Distribution of data among train, validation, text datasets

However here we did not try any balancing and as a result we had a very imbalanced set of 1–5-star reviews (marked as 0–4 for model), with lots of examples for 5 starts, fewer for 1 and 4 stars, and practically no examples for other classes:

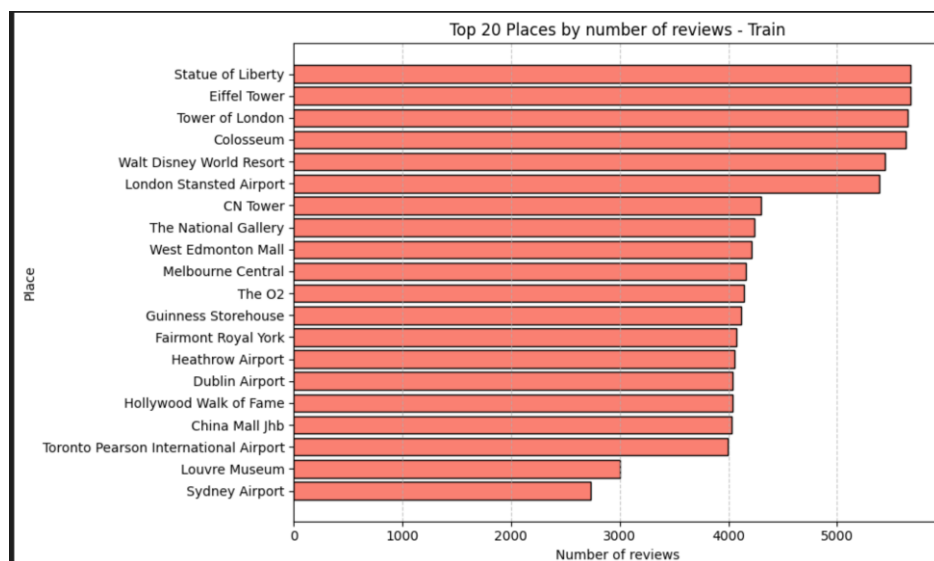


2.2.2.2. picture. Unbalanced distribution in full set

After even more careful analysis we managed to conclude that we might have problems with truncation negative reviews since they are statistically longer than positive ones and that our model might be better at recognizing hotel, restaurant and airport review better than of any other place:



2.2.2.3. picture. Average review length of each label



2.2.2.4. picture. Top 20 places of reviews in full dataset

We also analyzed the most popular words in each class to understand, what might be a signal of good or bad experience for model. Judging by results we assume our model might be more likely to identify any airport reviews as worse ones, than they actually are, but words like great and amazing might make model biased towards thinking that reviews is more positive.

Rating 0	Rating 1	Rating 2	Rating 3	Rating 4
airport (4470)	get (1336)	get (1619)	good (3176)	great (11361)
get (3495)	people (1095)	good (1229)	great (3011)	place (9149)
one (3302)	airport (1076)	people (1149)	get (2631)	see (6710)
people (3110)	one (913)	airport (1105)	place (2178)	get (6603)
...				
like (2450)	security (743)	go (915)	go (1604)	amazing (5550)
security (2280)	even (742)	would (856)	see (1582)	go (5298)

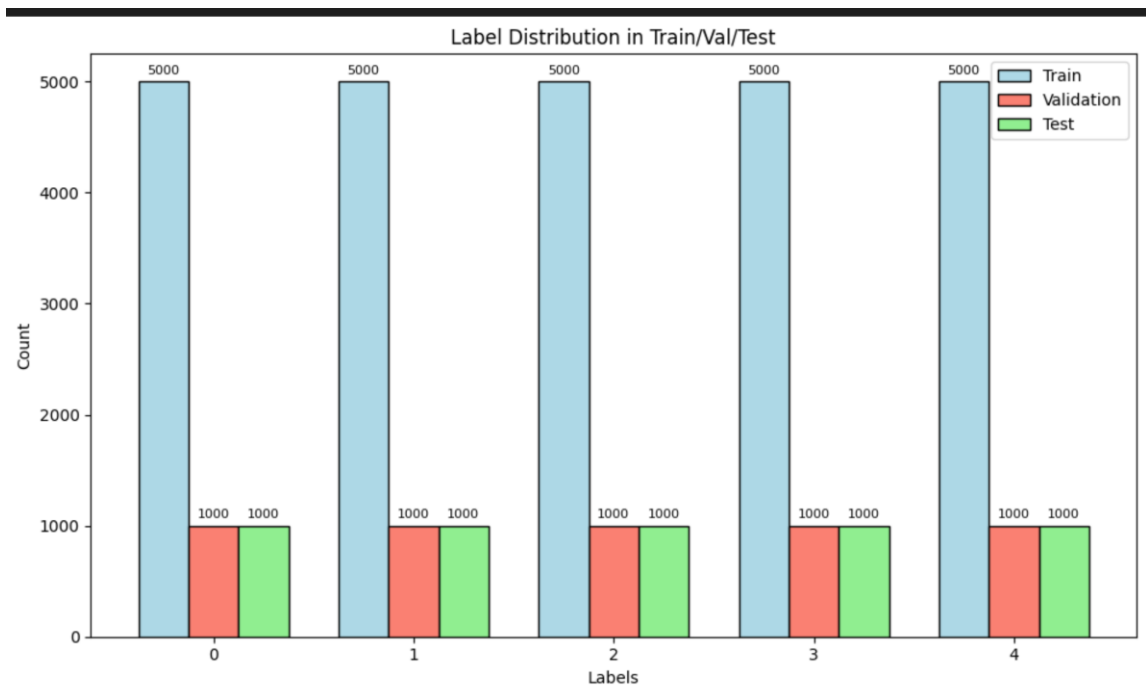
2.2.2.5. Most popular words for each rating

2.2.3. 25/5/5 split approach

The 25/5/5 approach refers to a deliberate splitting of the dataset into training, validation, and test sets with a specific ratio: 25k sequences for training, 5k for validation, and 5k for testing. This splitting is particularly useful since it allows both balancing and small size. With this set, we wanted to test if we can achieve a good performance by both balancing and shrinking the amount of data used.

```
Train size: 25000 (18.5%)  
Validation size: 5000 (3.7%)  
Test size: 5000 (3.7%)
```

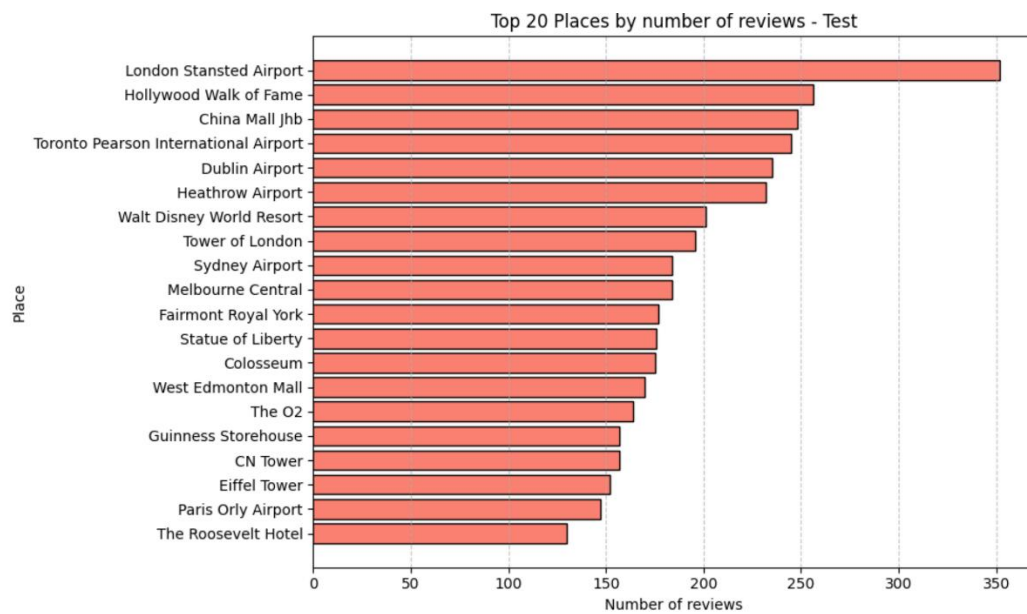
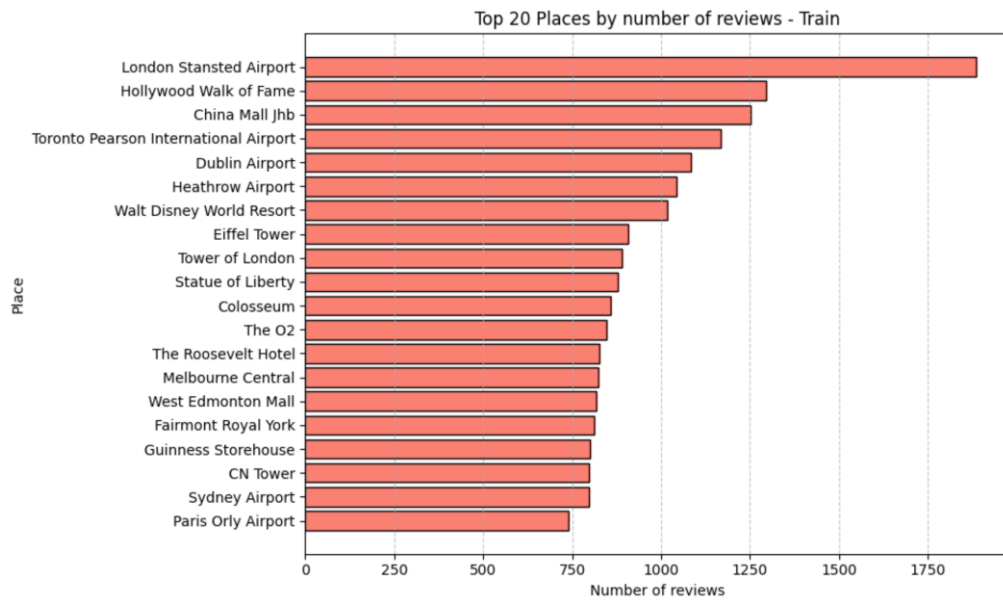
2.2.3.1. picture. Distribution of data among train, validation, text datasets in 25/5/5 splitting (percent of full dataset)



2.2.3.2. picture. Balanced distribution in 25/5/5 set

This split, however, led to an unexpected situation where airport reviews became dominant across all subsets (train, validation, and test). Consequently, the most frequent words among negative samples remained largely unchanged, which may impact the final F1 score, particularly in terms of generalizability to other public place types. While this class imbalance could be perceived as a limitation, especially given our original goal to model sentiment across various public places, we chose not to artificially reduce or remove the airport reviews for the following reasons:

- **Real-World distribution:** The dominance of airport reviews reflects the natural frequency and availability of user feedback on different public spaces.
- **Model robustness:** Retaining the imbalance allows us to test the model's ability to generalize despite skewed data.



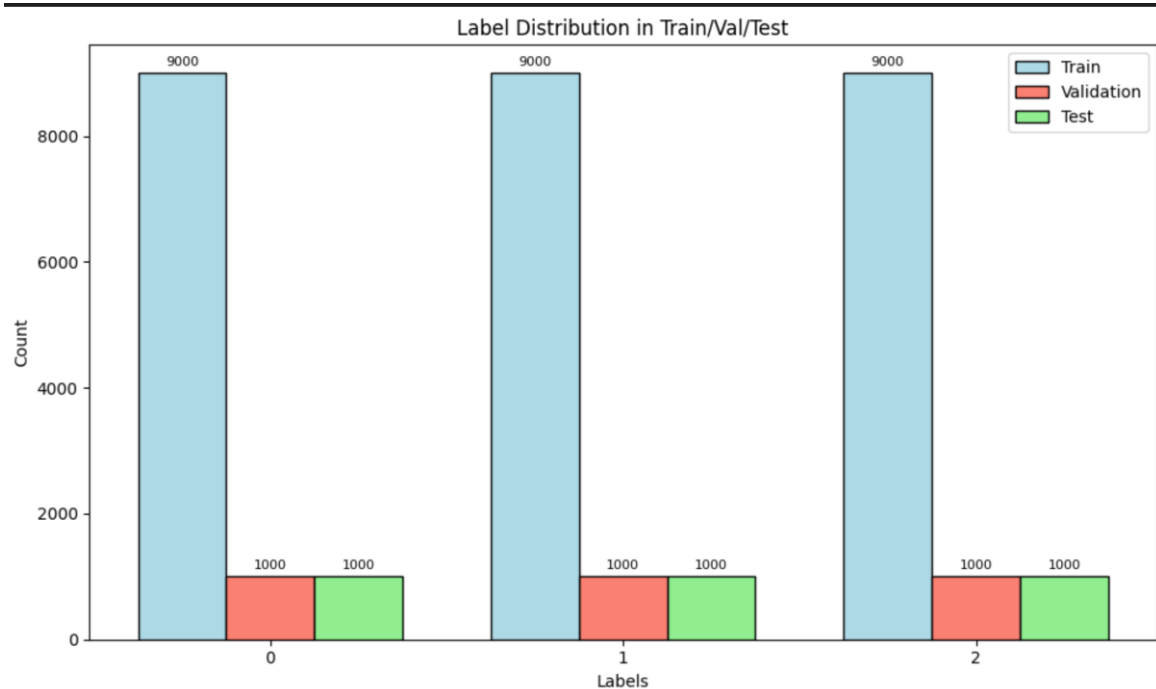
2.2.3.3. picture. Top 20 places of reviews in 25/5/5 split train and validation sets

2.2.4. 3-classes approach

To further analyze model performance, the original multi-class labeling was condensed into a 3-class classification problem. This re-labeling grouped similar categories - positive (4-5 stars), negative (1-2 stars) and neutral (3 stars), together based on behavioral or diagnostic similarity. This simplification aimed to reduce label noise and improve classification robustness, particularly when the original class definitions had overlaps or subjective elements. In that approach each class had an equal split of 9k rows since we wanted to provide slightly more examples than 25/5/5 split, while making testing a bit faster:

```
Train size: 27000 (20.0%)
Validation size: 3000 (2.2%)
Test size: 3000 (2.2%)
```

2.2.4.1. picture. Distribution of data among train, validation, test datasets in 3 class (percent of full dataset)



2.2.3.2. picture. Balanced distribution in 3-class set



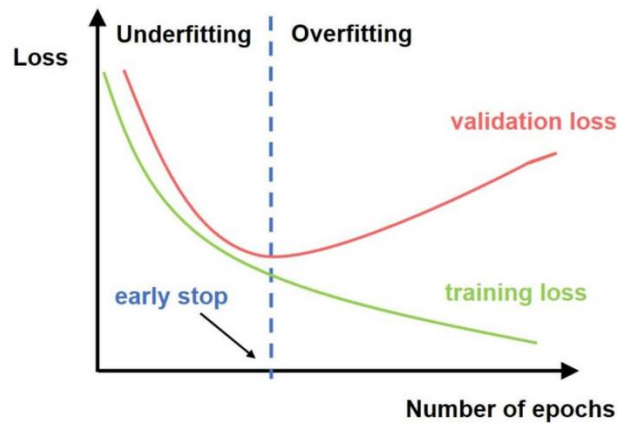
2.2.4.3. picture. Top-20 places in 3-class set

Unfortunately, it followed the same tendency with many airport data received in random splitting. However, it still doesn't mean that the model will be useless as explained before.

2.3. Metrics and optimization solutions used

2.3.1. Metrics for model evaluation

The core metrics for selecting the best model during training process was F1 score combined with training and validation loss. The best model was selected by best F1 and the smallest difference between training and validation loss (training loss was expected to be lower)



2.3.1.1. Picture. Approach we followed when training our models

We also used different metrics for model evaluation and comparison after training such as:

- **Accuracy:** To measure the proportion of correct predictions across all classes. While useful for a general overview, accuracy alone can be misleading.
- **Off-by-1 Accuracy:** This metric was used to consider predictions that are within one class distance from the correct label as acceptable. It is particularly useful in ordinal classification problems where classes have a natural order (e.g., rating scales), and a near-miss is less severe than a totally wrong prediction.
- **Macro Precision:** To calculate precision independently for each class and then takes the average. This treats all classes equally, regardless of their size. It is useful to understand how well the model avoids false positives across minority classes.
- **Macro Recall:** Similar to macro precision, but to evaluate how well the model identifies all relevant instances per class. This metric helps assess the model's ability to handle underrepresented categories, as each class contributes equally.
- **Macro F1-score:** The mean of macro precision and macro recall. It provides a balanced view when we care equally about false positives and false negatives across all classes, especially in imbalanced datasets.
- **Weighted Precision:** Like macro precision, but it weights the precision of each class by its number of true instances. This favors performance on more common classes, giving a more realistic view of how the model performs in practice.

- **Weighted Recall:** Weights recall by class frequency. This gives an estimate of how well the model performs on all instances, considering class imbalance.
- **Weighted F1-score:** Combines weighted precision and weighted recall into a single measure. It is especially informative for imbalanced datasets, as it reflects the overall performance while still penalizing poor performance in large classes.
- **Precision and recall for each class:** to understand model's performance on each class separately

2.3.2. Text processing

Since the BERT tokenizer handles most of the text preprocessing internally - including lowercasing, punctuation splitting, subword tokenization, and handling of out-of-vocabulary words - our preprocessing requirements were minimal. The only additional step we performed was whitespace normalization, such as removing extra spaces, to ensure cleaner input for the tokenizer.

```
check_bad_rows(reviews)

# Clean 'text' column: remove extra spaces/newlines
reviews['text'] = reviews['text'].str.strip()
reviews['text'] = reviews['text'].str.replace(r'\s+', ' ', regex=True)
✓ 2.4s

Empty rows found: 0

Rows with multiple spaces/newlines: 22008
```

2.3.2.1. picture. Whitespace removing on full dataset

2.3.3. Training and dataset flags

To have control over model training and data mode chosen we ensured it is provided by special flags in the beginning of our code. The only downside of our approach is that only one data mode can be true at a time to avoid errors.

```
1.3.1. Train data separation flags

# Data modes
# only one datamode at a time can be selected !!!

# Take full dataset of 100k reviews collected and 1-5 stars
FULL_SET = False

# Use sampler to ease the learning process, otherwise - cut 25k for training, 5k for test
SAMPLER = False and FULL_SET

# Use simplified labels - positive(2), neutral(1), negative(0)
THREE_CLASS = True
✓ 0.0s

1.3.2. Training and tokinezation flags flags

# Do we want to train at all
TRAIN = True

# finetune nlp-town bert model for 5 classes(1-5 stars)
TRAIN_BERT = False

# train distilbert from scratch for 5 classes(1-5 stars)
TRAIN_DISTILBERT = False

# train our handmade lstm model for 5 classes(1-5 stars)
TRAIN_RNN = True
```

2.3.3.1 picture. Data mode and training flags

2.3.4. Tokenization optimization

To optimize the efficiency of our pipeline, we saved the tokenized versions of the texts to disk. Since BERT tokenization is computationally expensive this approach significantly reduced redundant processing time, GPU usage, and overall computational cost during training and evaluation. By reusing the pre-tokenized inputs, we improved the reproducibility and scalability of our experiments without sacrificing performance.

```
if TOKENIZE:
    train_tokens = tokenize_texts(texts_train, tokenizer)
    val_tokens = tokenize_texts(texts_val, tokenizer)
    test_tokens = tokenize_texts(texts_test, tokenizer)

    train_tokens_distilbert = tokenize_texts(texts_train, tokenizer_distilbert)
    val_tokens_distilbert = tokenize_texts(texts_val, tokenizer_distilbert)
    test_tokens_distilbert = tokenize_texts(texts_test, tokenizer_distilbert)

    # Save tokenized data to disk
    save_tokenized(train_tokens, 'train_tokens.pt')
    save_tokenized(val_tokens, 'val_tokens.pt')
    save_tokenized(test_tokens, 'test_tokens.pt')

    #TOKENIZE=False

else:
    # Load from files
    train_tokens = load_tokenized('train_tokens.pt')
    val_tokens = load_tokenized('val_tokens.pt')
    test_tokens = load_tokenized('test_tokens.pt')
```

✓ 14.1s

2.3.4.1. picture Tokenization optimization

2.3.5. Parallel training using local machines and Kaggle

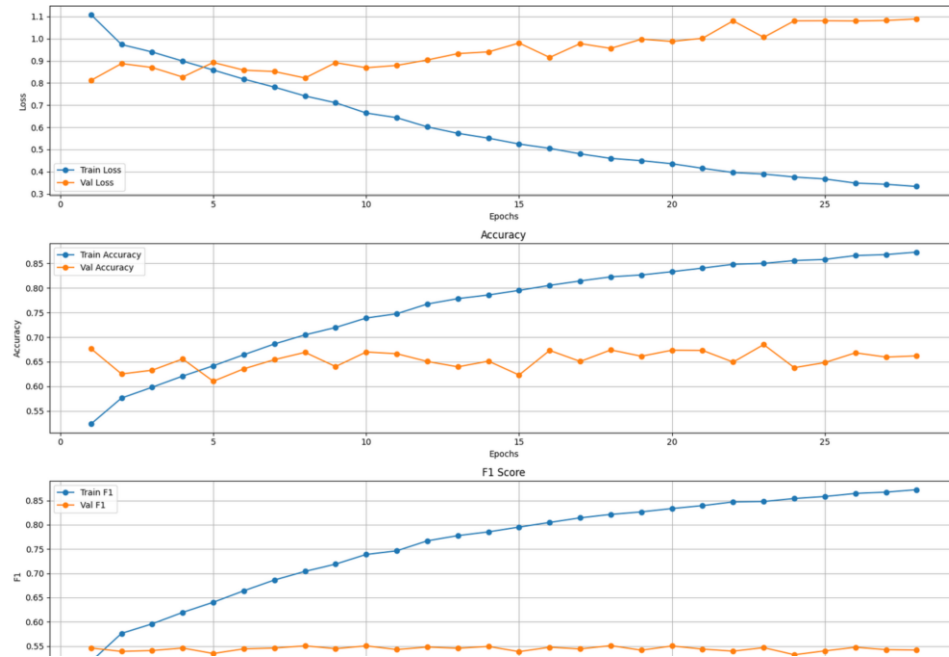
To accelerate model training and experiment with different configurations simultaneously, we adopted a parallel training strategy utilizing both local machines and Kaggle's cloud-based GPU environment. This hybrid setup allowed us to:

- **Distribute experiments** (e.g., testing different hyperparameters, data splits, or labeling strategies) across multiple devices, reducing total experimentation time.
- **Leverage Kaggle's free GPU resources**, such as T4s, to offload more computationally intensive training sessions without overloading local hardware.
- **Run lighter or debugging tasks locally**, ensuring faster iteration cycles for code testing and model validation.

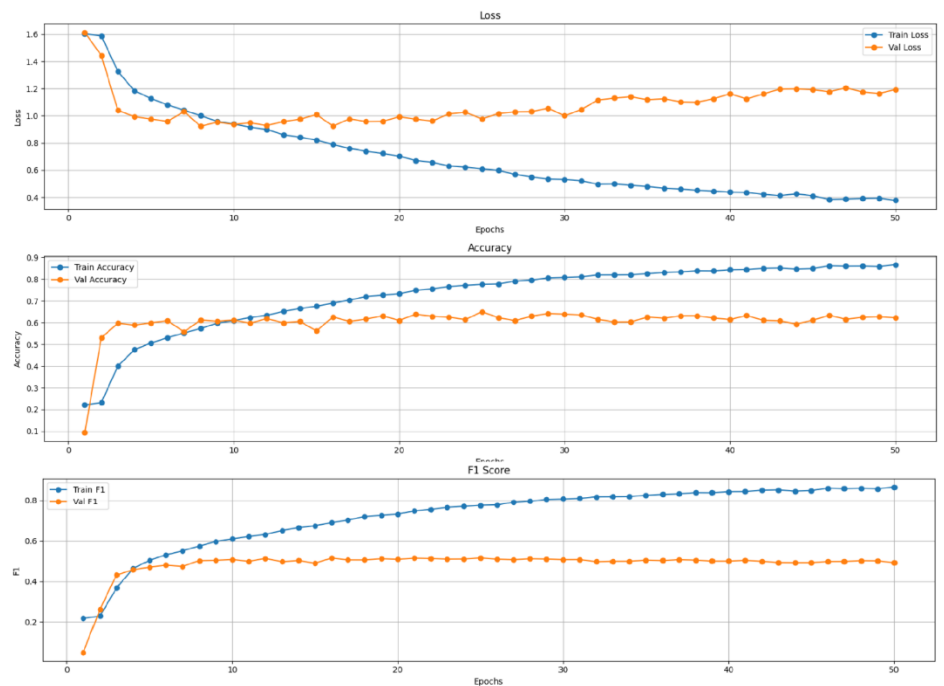
This approach not only improved overall efficiency but also allowed us to scale up experimentation without requiring dedicated cloud infrastructure.

3. RESULTS

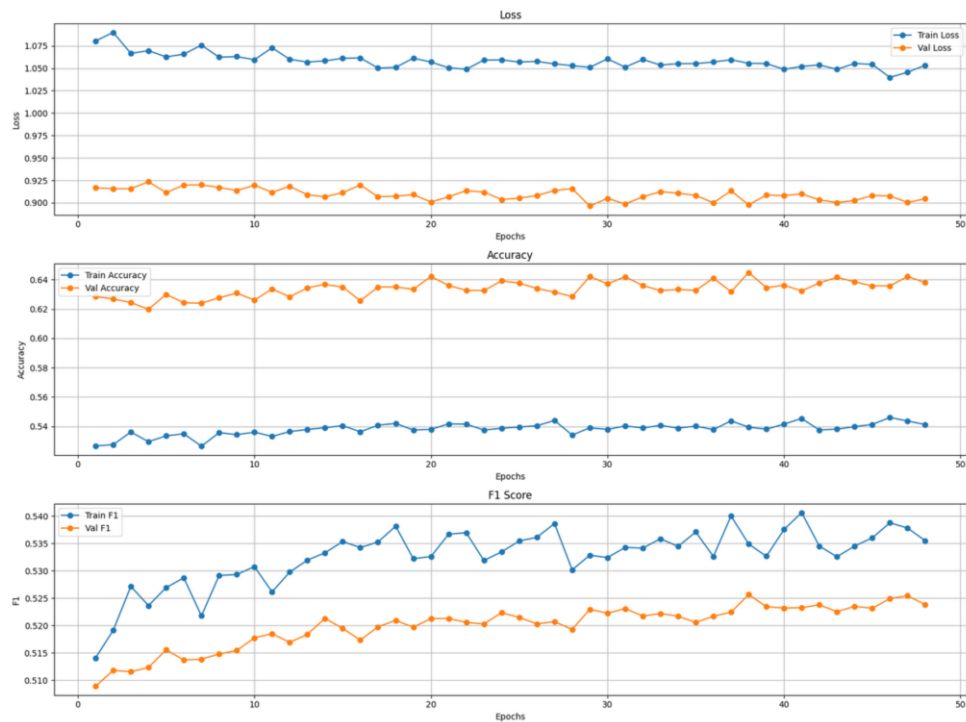
3.1. Sampler approach



3.1.1. picture. DistilBERT train/validation curves with sampler



3.1.2. picture. LSTM train/validation curves with sampler



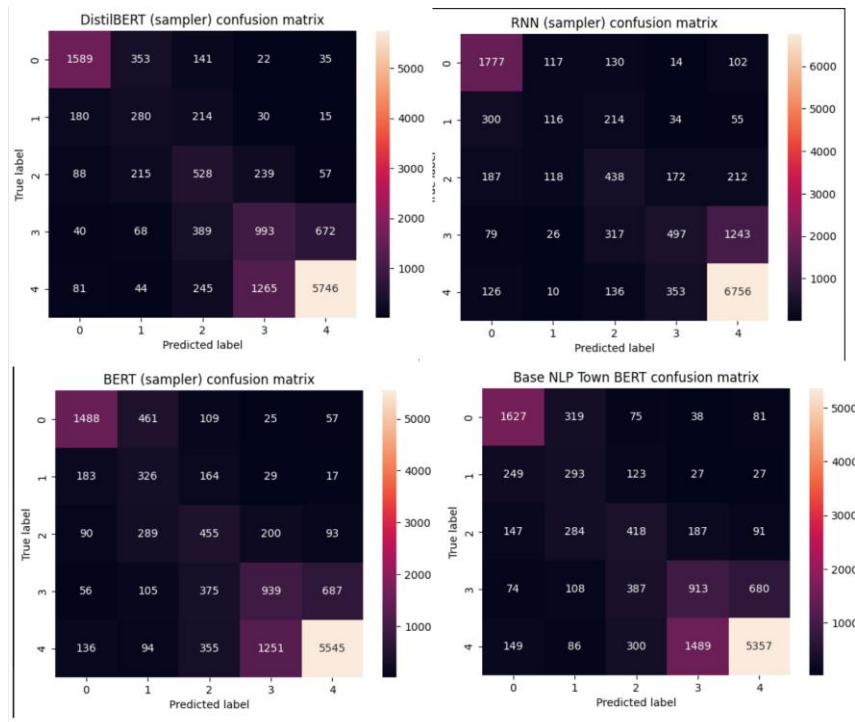
3.1.3. picture. BERT train/validation curves with sampler

The training curves allow us to conclude that while RNN model and DistilBERT were learning sufficiently with sampler, achieving saturation after 5-10 epochs, while BERT model struggled, even despite setting lower learning rates and using different optimizers such as AdamW. For it, the balanced number of unseen examples was not enough to extract enough information. We also can notice that it found validation set much easier, that again proofs our thoughts about too easy validation set and too unsaturated training set.

The average time taken for BERT and DistilBERT were 2-3 hours, while LSTM finished task in 7 minutes.

	Accuracy	Off-by-1 accuracy	Macro Precision	Macro Recall	Macro F1-score	Weighted Precision	Weighted Recall	Weighted F1-score
Model								
RNN (sampler)	0.708	0.918	0.529	0.505	0.503	0.674	0.708	0.681
BERT (sampler)	0.647	0.914	0.516	0.548	0.524	0.694	0.647	0.666
DistilBERT (sampler)	0.675	0.936	0.543	0.568	0.551	0.714	0.675	0.691
Base NLP Town BERT	0.636	0.911	0.503	0.537	0.515	0.679	0.636	0.653

3.1.4. picture. General classification report for sampler



3.1.5. picture. Confusion matrixes of sampler approach

RNN (sampler) classification report					DistilBERT (sampler) classification report				
	precision	recall	f1-score	support		precision	recall	f1-score	support
0	0.72	0.83	0.77	2140	0	0.80	0.74	0.77	2140
1	0.30	0.16	0.21	719	1	0.29	0.39	0.33	719
2	0.35	0.39	0.37	1127	2	0.35	0.47	0.40	1127
3	0.46	0.23	0.31	2162	3	0.39	0.46	0.42	2162
4	0.81	0.92	0.86	7381	4	0.88	0.78	0.83	7381
accuracy			0.71	13529	accuracy			0.68	13529
macro avg	0.53	0.51	0.50	13529	macro avg	0.54	0.57	0.55	13529
weighted avg	0.67	0.71	0.68	13529	weighted avg	0.71	0.68	0.69	13529

BERT (sampler) classification report					Base NLP Town BERT classification report				
	precision	recall	f1-score	support		precision	recall	f1-score	support
0	0.76	0.70	0.73	2140	0	0.72	0.76	0.74	2140
1	0.26	0.45	0.33	719	1	0.27	0.41	0.32	719
2	0.31	0.40	0.35	1127	2	0.32	0.37	0.34	1127
3	0.38	0.43	0.41	2162	3	0.34	0.42	0.38	2162
4	0.87	0.75	0.80	7381	4	0.86	0.73	0.79	7381
accuracy			0.65	13529	accuracy			0.64	13529
macro avg	0.52	0.55	0.52	13529	macro avg	0.50	0.54	0.52	13529
weighted avg	0.69	0.65	0.67	13529	weighted avg	0.68	0.64	0.65	13529

3.1.6. picture. Reports by class of sampler

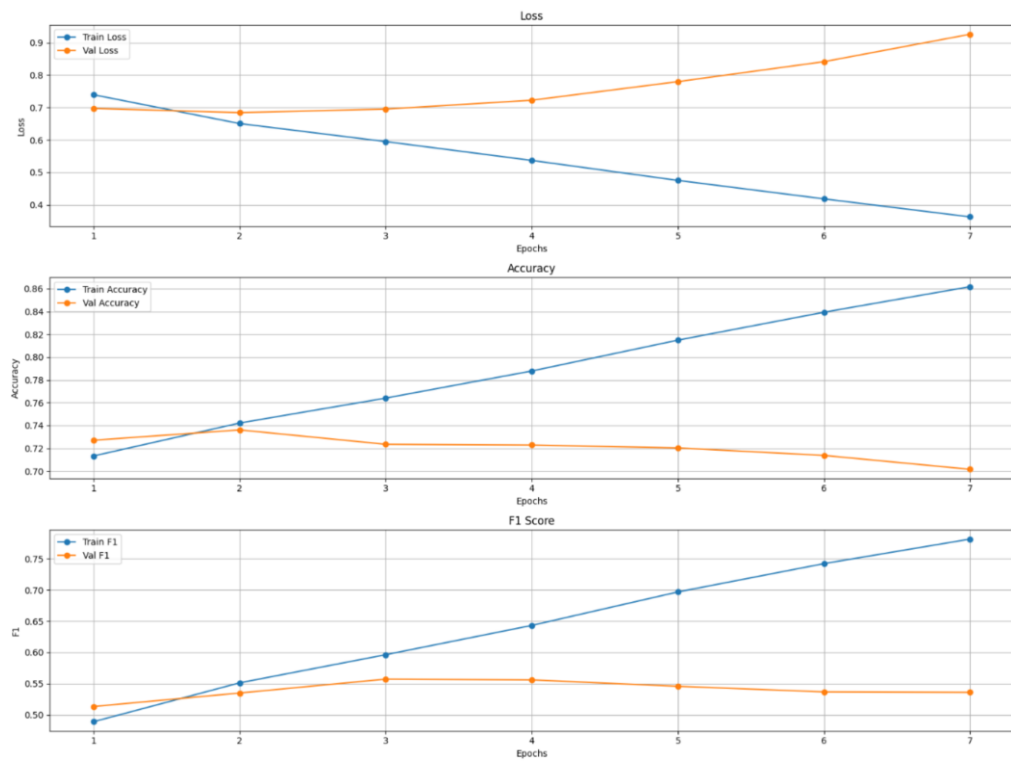
Among the four models - RNN (sampler), BERT (sampler), DistilBERT (sampler), and Base NLP Town BERT - the RNN (sampler) model achieved the highest overall accuracy at 0.708. DistilBERT (sampler) followed closely with an accuracy of 0.675. In contrast, the Base NLP Town BERT model showed the lowest accuracy at 0.636, indicating it was the least effective in predicting the correct class labels.

A closer examination of the confusion matrices reveals that all models performed best on class 4, showing high true positive counts and relatively low confusion with other classes. This suggests either an overrepresentation of class 4 in the dataset or particularly distinguishable features for this class. Additionally, a pattern of confusion between adjacent classes-such as classes 2 and 3 or 3 and 4- is present in all models, highlighting challenges in distinguishing between sentiment categories or

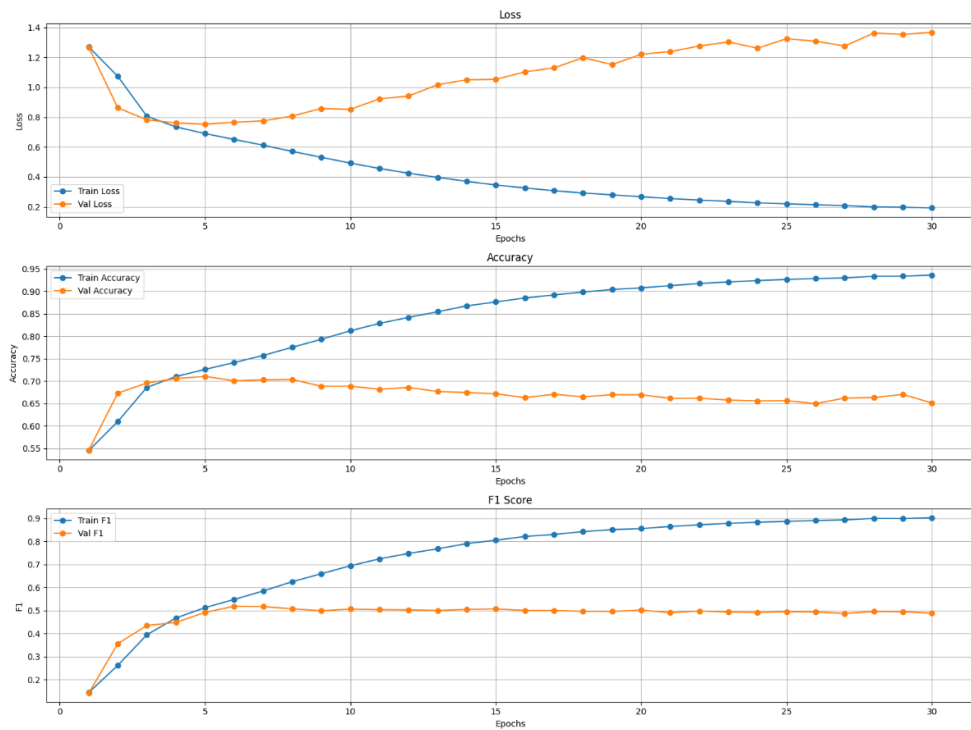
related labels that are numerically close. This observation is supported by the Off-by-1 accuracy scores, which are consistently high across all models, with DistilBERT (sampler) achieving the highest at 0.936.

When considering macro and weighted performance metrics, DistilBERT (sampler) emerges as the most balanced model. It achieved the highest macro F1-score (0.551) and weighted F1-score (0.691), suggesting strong performance across both majority and minority classes. In contrast, the RNN (sampler), despite its high exact accuracy, had a lower macro F1-score of 0.503, indicating potential bias toward majority classes. The BERT (sampler) model offered intermediate performance, with reasonably consistent metrics but slightly trailing DistilBERT. Meanwhile, Base NLP Town BERT underperformed across nearly all macro and weighted metrics, including the lowest macro precision (0.503) and weighted recall (0.636), reflecting weaker overall generalization.

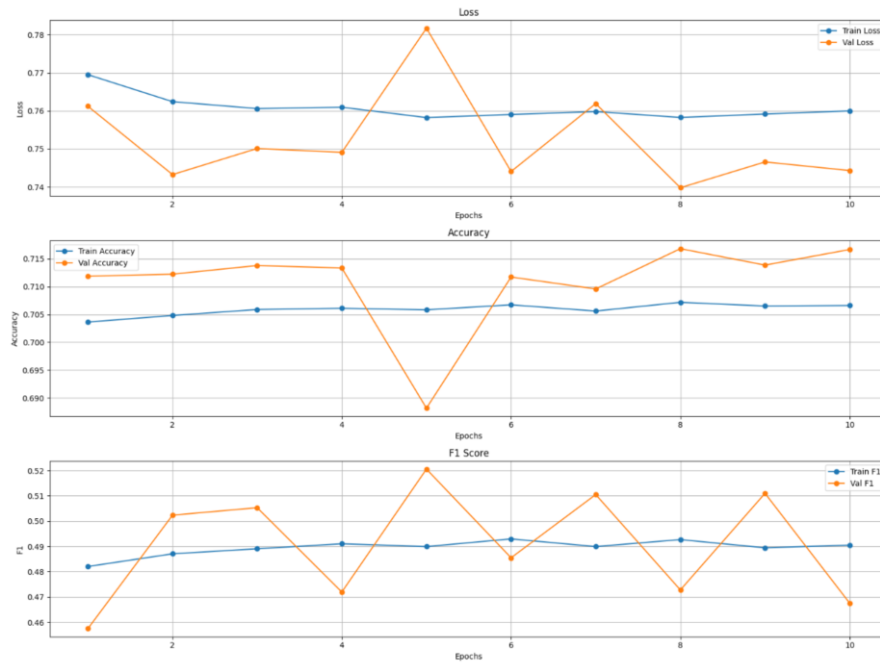
3.2. Full dataset approach



3.2.1. picture. DistilBERT train/validation curves with full set



3.2.2. picture. LSTM train/validation curves with full set

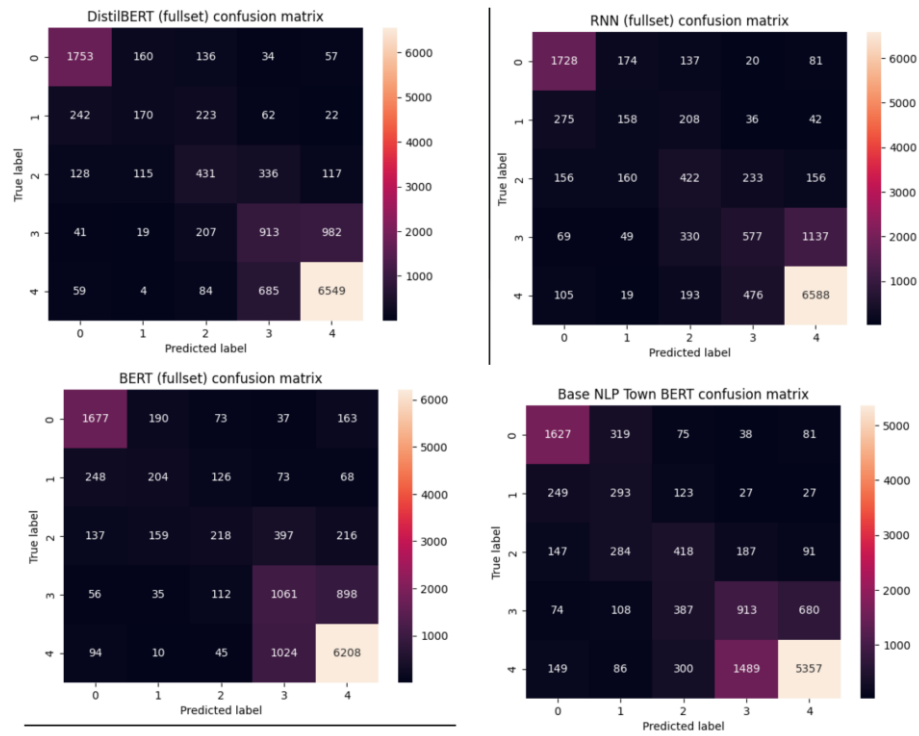


3.2.3. picture. BERT train/validation curves with full set

Without sampler we have achieved much faster saturation for DistilBERT and RNN model with 2 and 7 epochs accordingly and time of 40 minutes and 2 hours. BERT model also started to demonstrate better tendencies with more stable learning, still experiencing some fluctuations. Unfortunately, due to computing recourse limits, we were unable to train it for a longer period to get a clearer picture

	Accuracy	Off-by-1 accuracy	Macro Precision	Macro Recall	Macro F1-score	Weighted Precision	Weighted Recall	Weighted F1-score
Model								
RNN (fullset)	0.700	0.921	0.521	0.512	0.511	0.677	0.700	0.684
BERT (fullset)	0.692	0.926	0.542	0.519	0.523	0.684	0.692	0.685
DistilBERT (fullset)	0.726	0.944	0.570	0.550	0.557	0.712	0.726	0.717
Base NLP Town BERT	0.636	0.911	0.503	0.537	0.515	0.679	0.636	0.653

3.2.4. picture. General classification report for full set



3.2.5. picture. Confusion matrixes of full set

RNN (fullset) classification report					DistilBERT (fullset) classification report				
	precision	recall	f1-score	support		precision	recall	f1-score	support
0	0.74	0.81	0.77	2140	0	0.79	0.82	0.80	2140
1	0.28	0.22	0.25	719	1	0.36	0.24	0.29	719
2	0.33	0.37	0.35	1127	2	0.40	0.38	0.39	1127
3	0.43	0.27	0.33	2162	3	0.45	0.42	0.44	2162
4	0.82	0.89	0.86	7381	4	0.85	0.89	0.87	7381
accuracy			0.70	13529	accuracy			0.73	13529
macro avg	0.52	0.51	0.51	13529	macro avg	0.57	0.55	0.56	13529
weighted avg	0.68	0.70	0.68	13529	weighted avg	0.71	0.73	0.72	13529

BERT (fullset) classification report					Base NLP Town BERT classification report				
	precision	recall	f1-score	support		precision	recall	f1-score	support
0	0.76	0.78	0.77	2140	0	0.72	0.76	0.74	2140
1	0.34	0.28	0.31	719	1	0.27	0.41	0.32	719
2	0.38	0.19	0.26	1127	2	0.32	0.37	0.34	1127
3	0.41	0.49	0.45	2162	3	0.34	0.42	0.38	2162
4	0.82	0.84	0.83	7381	4	0.86	0.73	0.79	7381
accuracy			0.69	13529	accuracy			0.64	13529
macro avg	0.54	0.52	0.52	13529	macro avg	0.50	0.54	0.52	13529
weighted avg	0.68	0.69	0.68	13529	weighted avg	0.68	0.64	0.65	13529

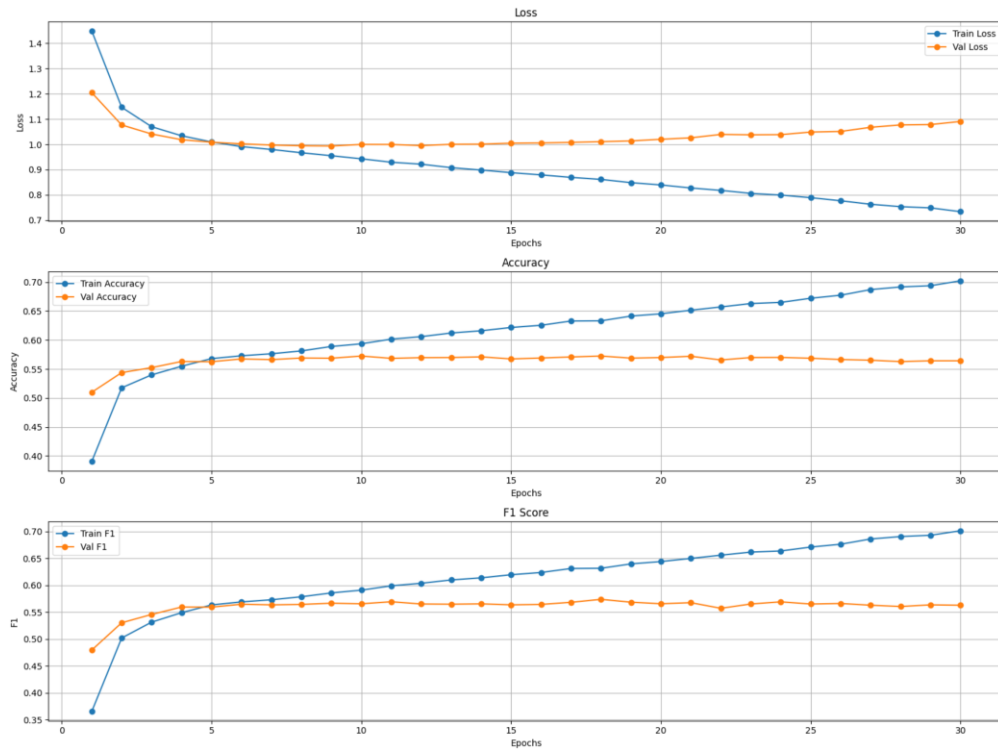
3.3.6. picture. Reports by class of full set

When evaluated on the full dataset without any sampling, DistilBERT clearly outperforms the other models across nearly all metrics. It achieves the highest overall accuracy (0.726) and leads in macro F1-score (0.557) and weighted F1-score (0.717), indicating a strong ability to generalize across both frequent and infrequent classes. Compared to its sampler counterpart (accuracy 0.675, macro F1-score 0.551), full-set DistilBERT shows notable improvements in both precision and recall, suggesting that removing sampling may have benefited the model by exposing it to the full class distribution.

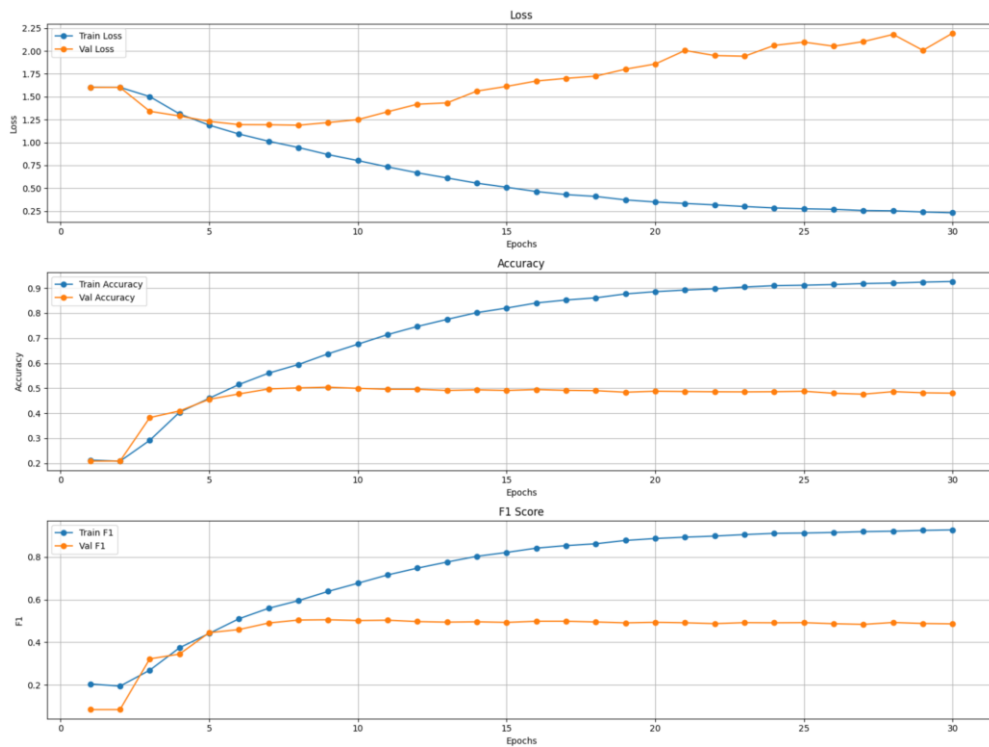
The RNN model maintains strong performance with an accuracy of 0.700 and Off-by-1 accuracy of 0.921, but its macro F1-score (0.511) remains lower than DistilBERT, echoing previous concerns about its bias toward majority classes. Its performance is slightly reduced from the sampler version (accuracy 0.708), suggesting that while RNNs perform well overall, they may benefit more from class-balancing techniques.

BERT (fullset) achieves competitive results, with macro and weighted F1-scores of 0.523 and 0.685, respectively. While slightly behind DistilBERT, it still outperforms the Base NLP Town BERT. Interestingly, BERT's accuracy (0.692) is slightly lower than in the sampler version, but macro metrics are comparable, indicating stable class-level performance. Also, class 4 is consistently the easiest to predict, possibly due to overrepresentation or more distinct features.

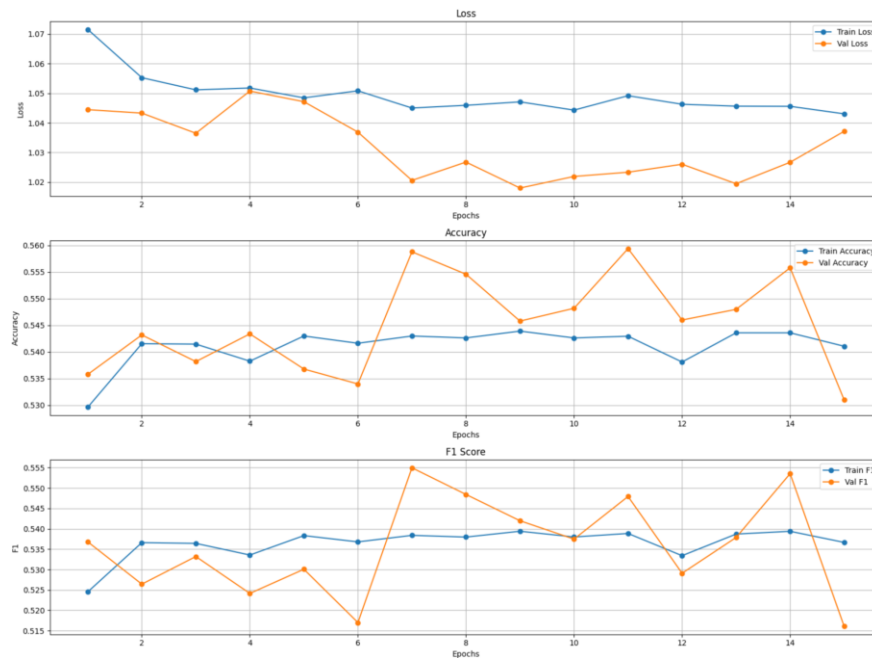
3.3. 25/5/5 split



3.3.1. picture. DistilBERT train/validation curves with 25/5/5 split



3.3.2. picture. LSTM train/validation curves with 25/5/5 split

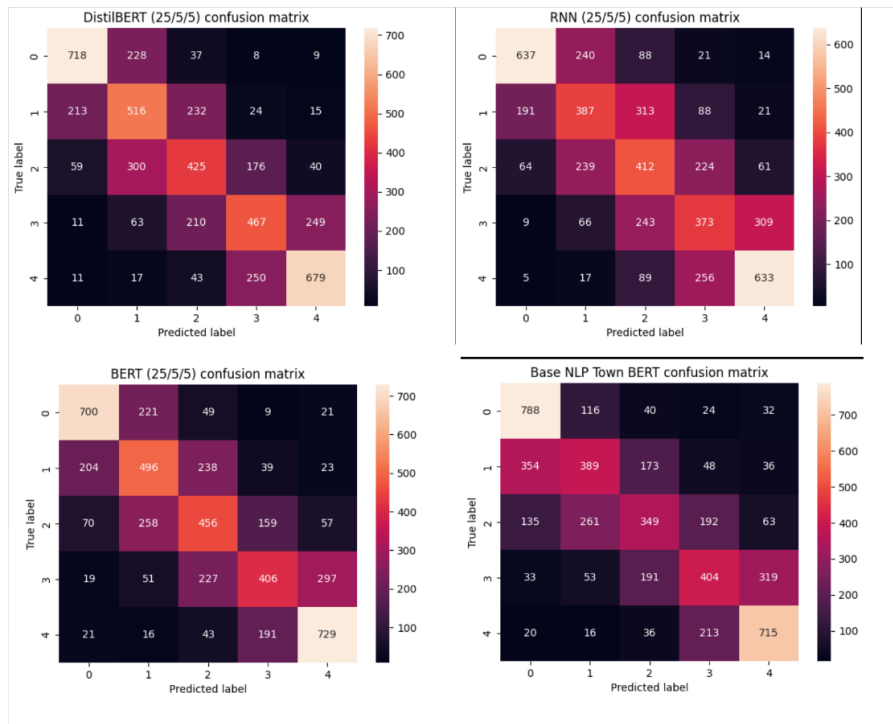


3.3.3. picture. BERT train/validation curves with 25/5/5 split

With only 25'000 data rows we have achieved similar trends for DistilBERT, BERT and RNN, all of them achieving enough saturation after 5-7 epochs. However, here we can see that RNN model starts struggling with F1 score, while BERT's family models do not fall down the 0.51 line.

	Accuracy	Off-by-1 accuracy	Macro Precision	Macro Recall	Macro F1-score	Weighted Precision	Weighted Recall	Weighted F1-score
Model								
RNN (25/5/5)	0.488	0.891	0.494	0.488	0.490	0.494	0.488	0.490
BERT (25/5/5)	0.557	0.916	0.554	0.557	0.554	0.554	0.557	0.554
DistilBERT (25/5/5)	0.561	0.933	0.561	0.561	0.561	0.561	0.561	0.561
Base NLP Town BERT	0.529	0.893	0.515	0.529	0.516	0.515	0.529	0.516

3.3.4. picture. General classification report for 25/5/5 split



3.3.5. picture. Confusion matrixes of 25/5/5 split

RNN (25/5/5) classification report					DistilBERT (25/5/5) classification report				
	precision	recall	f1-score	support		precision	recall	f1-score	support
0	0.70	0.64	0.67	1000	0	0.71	0.72	0.71	1000
1	0.41	0.39	0.40	1000	1	0.46	0.52	0.49	1000
2	0.36	0.41	0.38	1000	2	0.45	0.42	0.44	1000
3	0.39	0.37	0.38	1000	3	0.50	0.47	0.49	1000
4	0.61	0.63	0.62	1000	4	0.68	0.68	0.68	1000
accuracy			0.49	5000	accuracy			0.56	5000
macro avg	0.49	0.49	0.49	5000	macro avg	0.56	0.56	0.56	5000
weighted avg	0.49	0.49	0.49	5000	weighted avg	0.56	0.56	0.56	5000

BERT (25/5/5) classification report					Base NLP Town BERT classification report				
	precision	recall	f1-score	support		precision	recall	f1-score	support
0	0.69	0.70	0.70	1000	0	0.59	0.79	0.68	1000
1	0.48	0.50	0.49	1000	1	0.47	0.39	0.42	1000
2	0.45	0.46	0.45	1000	2	0.44	0.35	0.39	1000
3	0.50	0.41	0.45	1000	3	0.46	0.40	0.43	1000
4	0.65	0.73	0.69	1000	4	0.61	0.71	0.66	1000
accuracy			0.56	5000	accuracy			0.53	5000
macro avg	0.55	0.56	0.55	5000	macro avg	0.51	0.53	0.52	5000
weighted avg	0.55	0.56	0.55	5000	weighted avg	0.51	0.53	0.52	5000

3.3.6. picture. Reports by class of 25/5/5 split

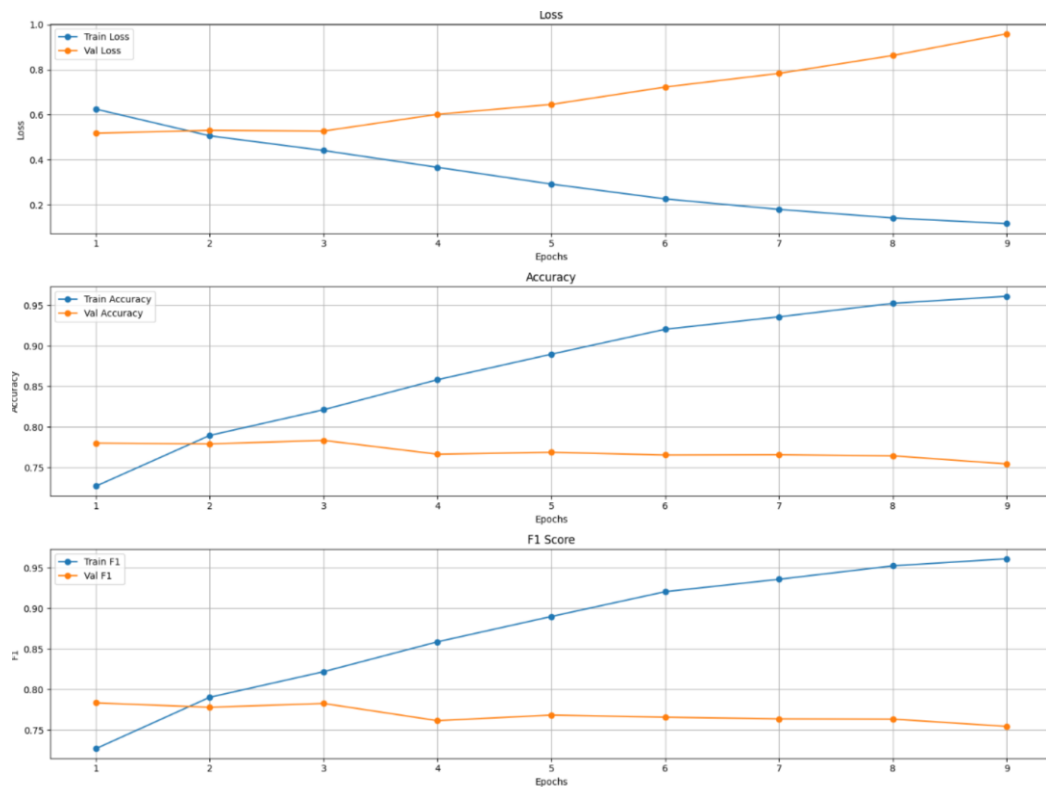
DistilBERT still emerges as the top performer among the models. It consistently achieves the highest metrics across the board, including an Accuracy of 0.561, Off-by-1 Accuracy of 0.933, and Macro/Weighted Precision, Recall, and F1-scores of 0.561. This indicates that DistilBERT is not only the most accurate in its direct predictions but also performs exceptionally well when considering predictions that are off by one label. Its confusion matrix also appears to have more concentrated diagonal values compared to other models, indicating fewer misclassifications.

The RNN model demonstrates the weakest performance. With an Accuracy of 0.488 and an Off-by-1 Accuracy of 0.891, it significantly stays behind the other models. Its Macro/Weighted Precision, Recall, and F1-scores are also the lowest at 0.490. The confusion matrix for RNN shows a more

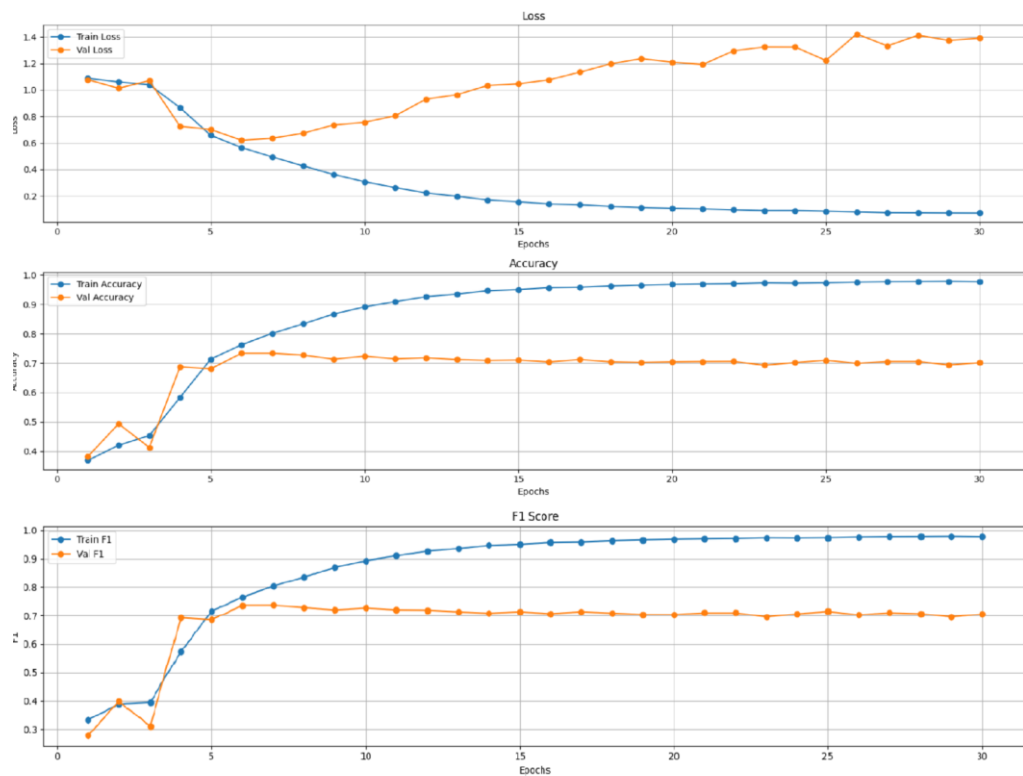
dispersed pattern, indicating a higher number of misclassifications across different classes, which aligns with its lower overall performance metrics.

A surprising aspect is how closely DistilBERT performs to the full BERT model, and in some cases, even surpasses it (e.g., DistilBERT's Off-by-1 accuracy is 0.933 vs BERT's 0.916). While BERT (25/5/5) shows strong performance with an Accuracy of 0.557 and corresponding F1-scores, DistilBERT, a smaller and faster version of BERT, manages to achieve slightly better or comparable results.

3.4. 3-class approach



3.4.1. picture. DistilBERT train/validation curves with 3-classes

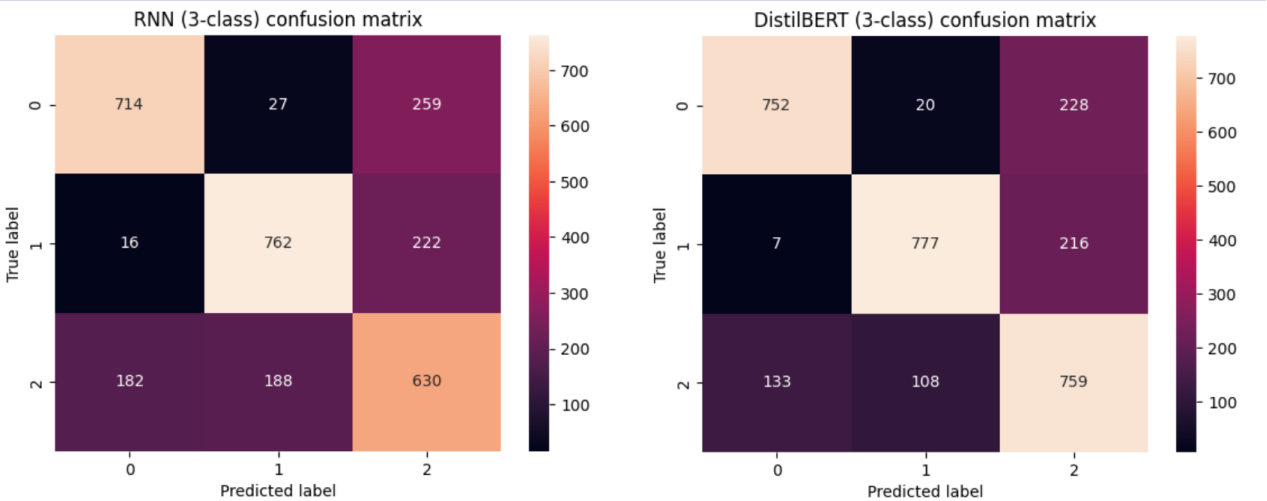


3.4.2. picture. LSTM train/validation curves with 3 class

After seeing high off-by-one tendencies, a decision of class union was taken and a 3-class models were trained. The DistilBERT achieved best results after 1 epoch (20 min), the RNN model - after 6th epoch (4 min). Both models demonstrate significant improvements of F1 score, making our decision about generalization. However, DistilBERT's training curves were exemplary, showcasing a rapid convergence of both training and validation losses, alongside stable and consistently high accuracy and F1 scores. This behavior strongly indicates superior generalization capabilities and a reduced tendency towards overfitting when compared to the RNN.

	Accuracy	Off-by-1 accuracy	Macro Precision	Macro Recall	Macro F1-score	Weighted Precision	Weighted Recall	Weighted F1-score
Model								
RNN (3-class)	0.702	0.853	0.710	0.702	0.705	0.710	0.702	0.705
DistilBERT (3-class)	0.763	0.880	0.778	0.763	0.767	0.778	0.763	0.767

3.4.3. picture. General classification report for 3 class



3.4.4. picture. Confusion matrixes of 3 class

RNN (3-class) classification report					
	precision	recall	f1-score	support	
0	0.78	0.71	0.75	1000	
1	0.78	0.76	0.77	1000	
2	0.57	0.63	0.60	1000	
accuracy			0.70	3000	
macro avg	0.71	0.70	0.70	3000	
weighted avg	0.71	0.70	0.70	3000	
DistilBERT (3-class) classification report					
	precision	recall	f1-score	support	
0	0.84	0.75	0.79	1000	
1	0.86	0.78	0.82	1000	
2	0.63	0.76	0.69	1000	
accuracy			0.76	3000	
macro avg	0.78	0.76	0.77	3000	
weighted avg	0.78	0.76	0.77	3000	

3.4.5. picture. Reports by class of 3-class

There has been great improvement across all models as they transitioned to the 3 class classification setup, where labels were re-mapped such that 0,1 became class 0, 2 became class 1, and 3,4 became class 2. Especially for the RNN (3-class) model, the improvement was immense, where its accuracy drastically increased to 0.702 and F1-scores to 0.705. Also, the confusion matrix showed marked improvement, particularly for class 1 and class 0, i.e., negative and neutral, which are not confused anymore. However, the analysis suggested that the misclassifications still remained prevalent between the extreme classes, i.e., 0 and 2, continuing to pose a challenge in distinguishing these groups with high confidence. On the other hand, DistilBERT (3-class) maintained first place and performed even better at an accuracy of 0.763 and F1-scores of 0.767. A very interesting observation in the confusion matrix confirmed the excellent concentration on the diagonal with very few misclassifications with DistilBERT.

In conclusion, the strategic re-mapping of labels undeniably simplified the classification task, leading to a notable boost in performance for both models. Throughout this analysis, DistilBERT consistently demonstrated itself to be more robust, generalize more effectively, and learn with greater efficiency, reaching its optimal performance rapidly. While the RNN did show improvement, it continued to exhibit limitations in distinguishing specific classes and displayed less stability in its training. Therefore, DistilBERT unequivocally stands as the superior choice for this particular classification problem.

3.5. Extra testing

	Accuracy	Off-by-1 accuracy	Macro Precision	Macro Recall	Macro F1-score	Weighted Precision	Weighted Recall	Weighted F1-score
Model								
Fullset DistilBERT	0.584	0.899	0.500	0.476	0.464	0.571	0.584	0.568
Sampler DistilBERT	0.490	0.881	0.413	0.431	0.418	0.520	0.490	0.501
25_SET DistilBERT	0.489	0.900	0.444	0.463	0.445	0.535	0.489	0.505

3.5.1. Picture. Comparing DistilBERTS on totally new dataset

Fullset DistilBERT classification report					
	precision	recall	f1-score	support	
0	0.40	0.60	0.48	80	
1	0.45	0.12	0.20	72	
2	0.42	0.38	0.40	172	
3	0.51	0.45	0.48	316	
4	0.72	0.82	0.76	460	
accuracy			0.58	1100	
macro avg	0.50	0.48	0.46	1100	
weighted avg	0.57	0.58	0.57	1100	
Sampler DistilBERT classification report					
	precision	recall	f1-score	support	
0	0.40	0.53	0.45	80	
1	0.19	0.21	0.20	72	
2	0.31	0.38	0.34	172	
3	0.42	0.45	0.43	316	
4	0.74	0.60	0.66	460	
accuracy			0.49	1100	
macro avg	0.41	0.43	0.42	1100	
weighted avg	0.52	0.49	0.50	1100	
25_SET DistilBERT classification report					
	precision	recall	f1-score	support	
0	0.49	0.47	0.48	80	
1	0.24	0.42	0.30	72	
2	0.34	0.43	0.38	172	
3	0.40	0.41	0.41	316	
4	0.75	0.58	0.65	460	
accuracy			0.49	1100	
macro avg	0.44	0.46	0.44	1100	
weighted avg	0.54	0.49	0.50	1100	

3.5.2. Picture. Comparing DistilBERTS on totally new dataset by classes

An extra test with found google maps restaurant review was used to understand, how model will perform on totally out of place data and interesting tendency was observed – the fullset model indeed showed the best results of all by all metrics. This suggests that leaving more data as is might be indeed better for general cases, than sampling or cutting off.

4. CONCLUSIONS

Overall, fine-tuning is generally considered a strong approach for NLP tasks. Still, our work demonstrates that fine-tuning BERT is not always the best choice. While it can produce good statistical results, it often requires significantly more resources for training and tokenization, especially for unbalanced datasets, to achieve good performance. In our case, even when fine-tuning was possible, we lacked sufficient data to make it truly effective. On the other hand, smaller models like DistilBERT offered a better balance between performance and efficiency, while LSTM models delivered results up to 10 times faster overall.

When it comes to data model selection, we recommend training on the full dataset without aggressive sampling or edge cutting. This approach allows models to extract more features and capture a fuller picture, resulting in better metrics. However, if precise classification is not the priority and resource efficiency is important, using a smaller dataset with merged labels paired with lightweight models can be an effective solution.

Regarding the data itself, a few improvements could be made. First, the category balance needs refinement. Instead of general "public/tourist places," categories should be more specific such as "restaurants," "airports," etc. Merging all categories into one is not ideal. Additionally, when using smaller models, category balancing becomes crucial. For instance, if analyzing both restaurants and hotels, the dataset should include an equal number of reviews for each. If focusing solely on restaurants, examples should be evenly distributed across different establishments. The more balanced the data, the better the model's performance.

We do not believe our current five-class classification DistilBERT and LSTM models are suitable for real-life deployment in their present form. However, it shows strong potential as a spam or inconsistency detector, especially in business-specific scenarios-such as identifying positive language in negative reviews or vice versa. With more training and access to cleaner, more structured data, our model could evolve significantly. In the future, it could also be applied to other domains such as sentiment analysis in customer support, trend detection in social media, or anomaly spotting in review platforms.